

What are CI CD pipelines ?

A **CI/CD pipeline** is a set of automated steps that help developers deliver code changes quickly and safely. “CI/CD” stands for **Continuous Integration** and **Continuous Delivery/Deployment**, and it’s a core practice in DevOps.

Continuous Integration (CI)

This part focuses on **frequent code integration**:

- Developers regularly push code to a shared repository (like GitHub).
- Automated tools **build** the application and run **tests**.
- Problems (bugs, conflicts) are detected early.

Example: Every time you push code, a system like Jenkins or GitHub Actions automatically checks if your code works.

Continuous Delivery (CD)

This ensures your code is always ready to be released:

- After passing tests, code is packaged and prepared for deployment.
- It may still require a **manual approval** to go live

Continuous Deployment (CD)

This goes one step further:

- Every change that passes tests is **automatically deployed** to production.
- No manual intervention is needed.

How a CI/CD pipeline works (simplified)

1. Developer pushes code
2. Pipeline triggers automatically
3. Code is built (compiled)

4. Automated tests run
5. If successful → deploy to staging or production

Why CI/CD pipelines matter

- Faster development and releases
- Fewer bugs in production
- Consistent, repeatable process
- Better collaboration across teams

When an application or product “goes live,” it means the software is moved from the development/testing environment to a real production environment where actual users can access it.

Here’s the complete flow used in most companies using DevOps and Agile.



How an Application Goes Live

1 Requirement Gathering

Business team or client gives requirements.

Example:

- Amazon wants a new “UPI Payment” feature.

Documents created:

- User Stories
 - Requirements
 - Acceptance Criteria
-

2 Planning & Task Creation

In Agile/Scrum:

- Product Owner creates features and user stories.
- Tasks are created in tools like:
 - Jira
 - Trello

Example:

- Epic → Payment Module
 - Feature → UPI Payment
 - User Story → “As a user, I want to pay using UPI”
-

3 Development Phase

Developers write code.

They use:

- IDEs like Visual Studio Code
- Git for version control
- Repositories like GitHub

Developer workflow:

Write Code → Commit → Push to Repository

4 CI Pipeline Starts

Once code is pushed:

CI/CD tools automatically trigger:

- Jenkins
- GitLab
- GitHub Actions
- Azure DevOps

Pipeline steps:

Pull Code



Build Application



Run Unit Tests



Code Quality Check



Create Deployable Package

If tests fail:

✗ Deployment stops.

If tests pass:

✓ Move to next stage.

5 Testing Environment (QA/UAT)

Application is deployed to testing servers.

Types of testing:

- Functional Testing
- Integration Testing
- Performance Testing
- Security Testing
- User Acceptance Testing (UAT)

Teams involved:

- QA testers
 - Business users
 - Clients
-

6 Approval Phase

Before production:

- Manager/Product Owner approves release.
- Release notes are prepared.

Sometimes:

- CAB approval (Change Advisory Board)
 - Security approval
-

7 Production Deployment (GO LIVE 🚀)

Now application is deployed to production servers.

Deployment methods:

- Manual Deployment
- Automated Deployment via CD pipeline

Common platforms:

- Amazon Web Services
 - Microsoft Azure
 - Google Cloud
-

8 Monitoring After Go Live

After deployment:

- Logs are monitored
- Errors checked
- Performance monitored

Monitoring tools:

- Splunk
 - Grafana
 - Datadog
-



Real Example Flow

Requirement



Planning



Coding



Git Push



CI Pipeline

(Build + Test)



QA Testing



Approval



CD Pipeline

(Deploy)



Production Live



Monitoring



Example: Amazon Payment Feature

Suppose Amazon adds “Cash on Delivery”:

Development:

- Frontend team builds UI
- Backend team creates APIs
- Database team updates tables

CI/CD:

- Code pushed to GitHub
- Jenkins pipeline runs tests
- QA tests payment flow

Go Live:

- Deployment to AWS production servers
- Users now see “Cash on Delivery”

Different Environments in a DevOps Project

1 Local Environment

This is the developer's personal machine.

Purpose

- Writing code
- Running application locally
- Debugging

Used By

- Developers

Example

A developer runs:

localhost:8080

2 Development Environment (DEV)

First shared environment after local development.

Purpose

- Integrate code from multiple developers
- Initial testing

Used By

- Developers

Characteristics

- Frequent deployments
- New features continuously added
- May contain bugs

Example

Deploy:

- Login module
 - Payment APIs
-

3 SIT Environment (System Integration Testing)

Environment for testing integration between systems/modules.

Purpose

Check whether:

- Frontend connects to backend
- Backend connects to database
- APIs communicate properly

Used By

- QA team
 - Integration testers
-

4 QA / Test Environment

Dedicated environment for testing.

Purpose

- Functional testing
- Regression testing
- API testing
- Bug validation

Used By

- QA engineers

Example Tests

- Login testing
 - Payment success/failure
 - Cart validation
-

5 UAT Environment (User Acceptance Testing)

Used for business/client validation.

Purpose

Ensure:

“Application meets business requirements.”

Used By

- Product Owner
- Clients
- Business users

Example

Business team validates:

- Payment workflow
 - UI behavior
 - Business rules
-

6 Staging Environment

A production-like environment.

Purpose

- Final testing before release
- Validate deployment process

Characteristics

- Same infrastructure as production
- Same server setup
- Same configurations

Used By

- DevOps engineers
 - Release teams
-

7 Production Environment (PROD)

The live environment.

Purpose

- Real users access the application

Used By

- Customers/end users

Characteristics

- Highly stable
- Highly secure
- Continuously monitored

Examples

- Amazon
- Netflix
- Flipkart

8 DR Environment (Disaster Recovery)

Backup environment for emergencies.

Purpose

Recover application if production fails.

Example

If production crashes:

- DR environment takes over.